Compte rendu TP2 – Récursivité : Diviser pour régner

Q.1)

```python
def dichotomie(L, x):
    n = len(L)-1
    debut = 0
    milieu = n//2
    fin = n
    nb_etapes=0

    while debut<=fin:
        milieu=(debut+fin)//2
        nb_etapes+=1
        if L[milieu]==x:
            return True, nb_etapes
        elif L[milieu]>x:
            fin = milieu-1
        else:
            debut = milieu + 1
    return False, nb_etapes
```

```python
def dichotomie_recursif(L, x, g=0, d=-1, nb_etapes=0):
    if d==-1:
        d=len(L)-1
    if g > d:
        return False,nb_etapes+1
    milieu = (g+d)//2
    nb_etapes+=1
    if L[milieu] == x :
        return True, nb_etapes
    elif L[milieu] > x :
        return dichotomie_recursif(L, x, g, milieu-1, nb_etapes)
    else :
        return dichotomie_recursif(L, x, milieu+1, d, nb_etapes)
```

Q.2)

```python
def triSimple(L):
    LL = []
    while len(L) > 0:
        m = L[0]
        for e in L:
            if e < m:
                m = e
        LL.append(m)
        L.remove(m)
    return LL
```

```python
def aleatoire(N):
    L = []
    for i in range(N):
        L.append(random.randint(0,100))
    return L

if __name__=='__main__':
    L = aleatoire(10)
    tic = timeit.default_timer()
    print(triSimple(L))
    print(timeit.default_timer()-tic)
```

```
tp2recursivite-13004502:~/tp2recursivite{main}$ /bin/python /home/user/tp2recursivite/tri.py
[0, 1, 2, 6, 9, 26, 31, 33, 52, 56, 61, 66, 71, 75, 79, 84, 85, 90, 94, 100]
8.83950001480116e-05
```

Q.3)

```python
def fusion(L1, L2):
    i = 0
    j = 0
    liste_fusion = []

    while i < len(L1) and j < len(L2):
        if L1[i] < L2[j]:
            liste_fusion.append(L1[i])
            i += 1
        else:
            liste_fusion.append(L2[j])
            j += 1
    while i < len(L1):
        liste_fusion.append(L1[i])
        i += 1
    while j < len(L2):
        liste_fusion.append(L2[j])
        j += 1
    return liste_fusion
```

```python
L1 = [1, 3, 5, 7]
L2 = [2, 4, 6, 8, 10]
print(fusion(L1, L2))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 10]
```

Q.4)

```python
def fusion_r(L1, L2):
    '''
    description : Implémentation de la partie fusion de l'algorithme de Tri-fusion
    entrée : L1 : une liste triée par ordre croissant
    sortie : L2 : une liste triée par ordre croissant
    sortie : la liste fusionnant les 2 listes L1 et L2 et triée par ordre croissant
    '''
    if len(L1) == 0:
        return L2
    elif len(L2) == 0:
        return L1
    elif L1[0] <= L2[0]:
        return [L1[0]] + fusion_r(L1[1:], L2)
    else:
        return [L2[0]] + fusion_r(L1, L2[1:])
```

```python
L1 = [1, 3, 5, 7]
L2 = [2, 4, 6, 8, 10]
print(fusion_r(L1, L2))
```

```
tp2recursivite-13004502:~/tp2recursivite{main}$ /bin/python /home/user/tp2recursivite/tri.py
[1, 2, 3, 4, 5, 6, 7, 8, 10]
```

Q.5)

```python
def triFusion(L):
    '''
    description : Implémentation de l'algorithme de Tri-fusion
    cette fonction est programmée de manière récursive avec appel
    à la fonction fusion de la première liste séparée en deux sous liste de
    même taille.
    entrée : L : une liste de valeurs à trier
    sortie : la liste triée
    '''
    if len(L) <= 1:
        return L
    else:
        return fusion_r(triFusion(L[:len(L)//2]), triFusion(L[len(L)//2:]))
```

```python
if __name__=='__main__':
    L = aleatoire(20)
    tic = timeit.default_timer()
    print(triFusion(L))
    print(timeit.default_timer()-tic)
```

```
tp2recursivite-13004502:~/tp2recursivite{main}$ /bin/python /home/user/tp2recursivite/tri.py
[8, 8, 10, 14, 20, 20, 20, 35, 35, 48, 55, 59, 72, 73, 76, 77, 87, 89, 98, 99]
9.257600004275446e-05
```

Q.6)

N=10 pour triSimple : `7.116699998732656e-05`

N=10 pour triFusion : `8.180799977708375e-05`

N=100 pour triSimple : `0.00023548599983769236`

N=100 pour triFusion : `0.00041196100028173 53`

N=1000 pour triSimple : `0.022707982000156335`

N=1000 pour triFusion : `0.016878583000107028`

Conclusion : Plus la liste est grande, plus le tri fusion est efficace par rapport au tri simple.